

Stoquify Agent Runtime Proposal Report

Executed prompt report | 2026-07-21

Generated: 2026-07-21

Folder: docs/agents-runtime

Scope: Execute the saved prompt STOQUIFY_AGENT_RUNTIME_REFINED_PROMPT_2026-07-21.md against the attached source document and current Stoquify workspace evidence.

Executive Recommendation

Stoquify should pursue the proposal, but only as a controlled Stoquify-native agent runtime built on top of existing protected services, read models, evidence trails, module entitlement, RBAC, redaction, snapshots, action queues, and workflow assurance. The right direction is not "AI chat inside every module." The right direction is a shared operating copilot layer that turns Stoquify's existing business truth into role-specific decisions, evidence-backed explanations, and safely routed actions.

This would move Stoquify from a business management SaaS with dashboards into a daily operating intelligence system for African and OHADA SMBs. The strongest value proposition is not novelty. It is frequency and trust: the owner, accountant, finance officer, HR specialist, payroll engineer, manager, stockkeeper, and POS cashier all have recurring questions that happen daily or hourly:

- What changed today?
- Where is cash blocked or missing?
- Which stock risks require action now?
- Which payment, payroll, close, or compliance item is not ready?
- Who is allowed to resolve it?
- What evidence supports the answer?
- What controlled workflow should happen next?

The implementation should start with three read-and-draft agents: Command Agent, Cash/Reconciliation Agent, and Inventory/Replenishment Agent. Exception handling and action orchestration should be platform infrastructure, not separate visible chatbots. Dangerous authority must remain human-owned and service-owned: no direct Prisma writes, no direct ledger posting, no statutory filing, no role changes, no payroll approval, no close certification, no cash adjustment, and no stock write-off by an agent.

Evidence Inspected

Local Stoquify evidence:

- package.json: Next.js 15, React 19, TypeScript/Prisma service architecture, Jest/Playwright tests, release gates, policy gates, workflow assurance gates, inventory, payment, purchasing, statutory, payroll, close, report-trust, and release-evidence scripts.
- graphify-out/GRAPH_REPORT.md: large repository graph with 4,121 nodes, 5,321 edges, and 135 communities, confirming broad module coverage.
- config/sidebar.ts: role/module-oriented navigation with Command, Operations, Finance & Trust, People, and Governance sections.
- services/_shared/protect.ts: shared server action protection with permission checks, tenant guard, module gate, fresh auth, safe errors, and correlation IDs.
- lib/security/rbac.ts: session-backed RBAC context, permission expansion, audit decisions, active organization enforcement, stale session organization detection, account lock, and email verification checks.
- services/modules/module-entitlement.service.ts and services/modules/module-control-contracts.ts: module catalog, entitlement decisions,

- observe/enforce modes, access intent, surface type, audit logging, and dependency gaps.
- services/signals/business-signal-contracts.ts, services/signals/business-signal.service.ts, and services/signals/action-queue.service.ts: existing business signals and action queue model with evidence grade, severity, assigned role, required permission, blockers, redactions, next action, and tenant filtering.
- services/snapshots/snapshot-contracts.ts and services/snapshots/tenant-operating-snapshot.service.ts: tenant operating truth, payment truth, inventory cash, close readiness, freshness, source hash, blockers, redactions, and source modules.
- services/payments/payment-reconciliation-workbench.service.ts: payment reconciliation read model with rails, failures, duplicate provider references, suspense readiness, and source limitations.
- services/snapshots/inventory-cash-snapshot.service.ts: inventory cash truth with stock quantities, stock value, zero/negative stock blockers, source freshness, and source modules.
- services/payroll/command-read-model.service.ts: payroll command read model with role scope, redaction, trusted counts, blockers, next actions, evidence, country-pack data, release readiness, and payroll payment evidence.
- services/accounting/close-assurance.service.ts: close assurance dashboard model with checklist, findings, evidence items, comments, reviews, controls, waiver fresh auth, and certification availability.
- services/accounting/posting.service.ts: ledger posting service with idempotency keys, source links, sensitive action control, transactions, audit events, and accounting invariants.
- services/security/step-up-auth.service.ts and services/security/redaction-policy.service.ts: step-up auth and sensitive-field redaction categories for payroll, supplier bank data, payment references, suspense, fiscal payloads, compliance submissions, audit context, close evidence, and exports.
- services/evidence/evidence-contracts.ts and components/evidence/ProofTrailDrawer.tsx: evidence grades, proof trail nodes, edges, blockers, redactions, next actions, and proof UI.
- Existing route surfaces under app/[locale]/(dashboard)/dashboard: daily digest, owner war room, manager action center, finance cash command, finance reconciliation, inventory, POS, accounting close, assurance control tower, payroll, compliance, settings modules, and more.

External sources checked:

- Vercel AI SDK 7 announcement and AI SDK tool calling docs: <https://vercel.com/blog/ai-sdk-7>, <https://ai-sdk.dev/docs/ai-sdk-core/tools-and-tool-calling>
- Model Context Protocol tools specification: <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>
- Inngest durable function docs: <https://www.inngest.com/docs/learn/inngest-functions>
- Trigger.dev task docs: <https://trigger.dev/docs/tasks/overview>
- Langfuse observability overview: <https://langfuse.com/?tab=observability>
- Helicone platform overview: <https://docs.helicone.ai/getting-started/platform-overview>
- LiteLLM getting started docs: <https://docs.litellm.ai/>

1. Document Interpretation

The attached document's core thesis is correct: Stoquify should not add random AI chat features. It should become an evidence-backed operating copilot that sits on top of its existing source of truth. The phrase "Here is what happened, here is what matters today, here is the evidence, here is the risk, here is the recommended action, and here is the controlled workflow to resolve it" is the best product framing.

The document makes five major points.

First, one shared agent runtime. This means Stoquify needs one internal control plane for all agents, not one framework per module. This is directly aligned with the existing architecture. The codebase already centralizes action protection in `services/_shared/protect.ts`, module entitlement in `services/modules`, RBAC in `lib/security`, snapshots in `services/snapshots`, and business signals in `services/signals`. The agent runtime should follow that same pattern.

Second, TypeScript-native building blocks. Stoquify is already a Next.js, TypeScript, Prisma system. The agent layer should be a TypeScript application concern, not a Python sidecar that gradually becomes a parallel product. The external research supports this: the AI SDK is now explicitly positioned for TypeScript agent work, tool calling, approvals, runtime context, MCP Apps, telemetry, and durability. That fits Stoquify's stack better than adopting a large Python-first agent framework.

Third, Stoquify-native domain skills. This is the most important strategic insight. Generic frameworks can provide model calls, tool schemas, execution loops, and traces. They cannot provide Stoquify's defensibility: tenant-scoped business truth, OHADA/SYSCOHADA context, RBAC, evidence grades, source hashes, stock-to-cash-to-close continuity, payroll redaction, close assurance, and controlled workflow execution. Those must remain Stoquify-owned.

Fourth, launch three user-facing agents first. The Command Agent, Cash/Reconciliation Agent, and Inventory/Replenishment Agent are the right starting set because they meet frequent operational needs and can reuse existing read models. They also avoid the highest-risk initial domains: statutory filing, ledger posting, close certification, payroll approval, and permission changes.

Fifth, no direct dangerous authority. This is non-negotiable. The proposal is valuable because agents explain, recommend, draft, and route actions. It becomes dangerous if agents directly mutate business records, post ledgers, approve payroll, certify close packs, file statutory documents, or change permissions.

2. Strategic Positioning

Executed well, this places Stoquify above ordinary SMB software categories.

Ordinary POS systems record sales and payments. Stoquify can explain cash truth: captured payments, drawer variances, open suspense, duplicate provider references, and reconciliation readiness.

Ordinary inventory tools show stock on hand. Stoquify can explain stock risk: negative levels, zero stock, overstock, dead stock cash exposure, stock-to-cash gaps, purchasing delays, and suggested replenishment.

Ordinary accounting tools show ledgers and reports. Stoquify can explain close readiness: which period is blocked, which checklist item failed, what evidence is missing, which source modules are trusted, and who must act.

Ordinary payroll systems calculate payslips. Stoquify can explain payroll readiness: missing contracts, attendance drift, payment destination gaps, declaration blockers, person-level redaction, country-pack provenance, and accounting close exposure.

Ordinary dashboards show metrics. Stoquify can become a role-specific operating layer that tells each user what needs attention now. This is a stronger category: an evidence-backed operating copilot for OHADA SMBs.

The positioning should be:

- Stoquify is not "AI for SMBs."
- Stoquify is not "a chatbot attached to ERP."

- Stoquify is not "dashboard insights."
- Stoquify is the trusted command system for SMB operations, finance, inventory, payroll, compliance, and close assurance.

That is a better market claim because it is tied to daily work, not abstract automation.

3. Technical Architecture

Stoquify should add a shared agent-runtime domain with a small number of tables and services. The runtime should not own business actions. It should own agent execution metadata, evidence binding, approvals, and safety.

Recommended Prisma model family:

- AgentDefinition: canonical agent key, title, status, owner module, default model policy, risk level, allowed tools, allowed skills, and rollout status.
- AgentSkillDefinition: Stoquify-owned skill key, version, domain, required permissions, redaction category, prompt template hash, and evaluation notes.
- AgentToolDefinition: tool key, service owner, input schema hash, output schema hash, risk level, approval policy, module slug, permission, and idempotency behavior.
- AgentRun: organizationId, actorId, agentKey, status, model, cost estimate, token counts, startedAt, completedAt, correlationId, source route, module decision snapshot, and failure metadata.
- AgentStep: runId, step number, kind, tool key, prompt hash, status, startedAt, completedAt, retry count, input hash, output hash, safe summary, and error category.
- AgentEvidenceLink: runId, stepId, subjectType, subjectId, sourceModule, sourceTable, sourceHash, evidenceGrade, freshness, redaction count, and proof-trail URI.
- AgentApproval: runId, tool key, action draft type, requestedBy, approvedBy, status, approval policy, freshAuthRequired, approvedAt, rejectedAt, expiration, and signed input hash.
- AgentActionDraft: runId, service action key, payload hash, visible summary, required permission, module slug, risk level, idempotency key, status, and execution result reference.
- AgentFeedback: runId, actorId, helpfulness, accepted, rejectedReason, correctionText, falsePositive, staleAnswer, unsafeAttempt, and follow-up outcome.
- AgentCostLedger: runId, model provider, model name, prompt tokens, completion tokens, tool cost, total estimated cost, currency, tenant attribution, and budget bucket.
- AgentPolicyIncident: runId, incident type, severity, blocked tool, policy name, details, resolvedBy, resolution status, and evidence.

The runtime services should be:

- agent-context.service.ts: resolves tenant, actor, role, permissions, locale, currency, time window, module entitlements, and available read models.
- agent-tool-registry.service.ts: exposes only service-owned tools with schemas, permission checks, risk level, and approval policy.
- agent-runner.service.ts: orchestrates model calls, tool loops, prompt versions, evidence collection, step logging, and safe error handling.
- agent-approval.service.ts: handles approval requests, fresh auth, signed input replay, expiration, and maker-checker rules.
- agent-evidence.service.ts: binds every answer and recommendation to snapshots, proof trails, source hashes, evidence grades, blockers, and redactions.
- agent-redaction.service.ts: wraps existing redaction-policy.service.ts and prevents sensitive output from leaking into prompts or responses.
- agent-action-draft.service.ts: creates drafts that route into existing protected service actions instead of executing dangerous writes directly.
- agent-observability.service.ts: records traces, costs, latency, model policy, feedback, and policy incidents.

This is intentionally small. Most value should come from wrapping existing services, not building a parallel AI business system.

4. Best-Fit Technology Stack

Vercel AI SDK or equivalent TypeScript-native agent loop

Recommendation: adopt first.

What it brings:

- TypeScript-native model calls and tool calling.
- Structured tool schemas using the same TypeScript/Zod style Stoquify already uses.
- Streaming responses for command surfaces.
- Tool-call lifecycle data that can map into AgentStep.
- Current support for agent production features such as approvals, typed tool context, runtime context, telemetry, and durable agent concepts.
- A natural fit for Next.js route handlers and React UI.

What it complicates:

- It introduces model-loop semantics into the app and requires strict tool boundaries.
- Approval behavior must still be mapped to Stoquify's own RBAC, fresh auth, and audit model.
- The app must avoid leaking sensitive data into prompts.

Verdict: worth using. It should be the first external building block because it gives the most leverage with the least code and fits the stack.

Model Context Protocol

Recommendation: design for it early, expose internally later.

What it brings:

- A standard way to expose tools with names, descriptions, input schemas, and tool invocation.
- A clean boundary between agents and Stoquify services.
- Potential future interoperability with other agent clients or internal admin copilots.
- Human-in-the-loop guidance aligns with Stoquify's safety requirements.

What it complicates:

- MCP is a protocol, not a governance model. Stoquify still needs RBAC, tenant isolation, redaction, evidence binding, approvals, and audit logs.
- Exposing too many tools too soon can create a large model-visible attack surface.
- Tool naming, versioning, and schema evolution must be tightly controlled.

Verdict: worth using as a tool boundary and future extension layer. In phase one, implement Stoquify's internal tool registry with MCP-compatible shapes before exposing a full MCP server.

Inngest-style durable execution

Recommendation: adopt when actions must pause, retry, wait, or resume.

What it brings:

- Durable, retrievable background functions.
- Step-level state and retries.
- Event, cron, and webhook triggers.

- Good fit for approval waits, reconciliation import processing, scheduled command digests, and long-running evidence rebuilds.

What it complicates:

- Adds another execution plane and operational dependency.
- Idempotency becomes mandatory for any write or external side effect.
- The team must decide which workflows are synchronous app requests versus durable jobs.

Verdict: worth using in phase two or three. It is not necessary for the first read-only Command Agent, but it becomes valuable for approvals, scheduled digests, retries, provider sync, and long-running workflow orchestration.

Trigger.dev-style durable tasks

Recommendation: evaluate side by side with Inngest before adoption.

What it brings:

- TypeScript tasks, retries, waits, and schema validation.
- Good developer ergonomics for background jobs and AI-related tasks.
- Straightforward task model for delayed and external callback workflows.

What it complicates:

- Similar category overlap with Inngest; Stoquify should choose one durable execution layer, not both.
- Requires task deployment discipline and separate operational monitoring.

Verdict: potentially worth using, but do not adopt both Inngest and Trigger.dev. Pick one after a short proof-of-concept around approval wait, retry, and idempotent service action execution.

Langfuse

Recommendation: adopt once agent traffic exists.

What it brings:

- Tracing for LLM calls, tool invocations, retrieval, cost, latency, and sessions.
- Prompt management, datasets, evaluations, experiments, and human feedback loops.
- Strong fit for measuring wrong answers, stale answers, unsafe tool attempts, and accepted recommendations.

What it complicates:

- Sensitive prompt and output data must be filtered, redacted, or self-hosted according to Stoquify policy.
- Adds an observability data store and privacy controls.

Verdict: worth using for agent observability and evaluations after the first internal read-only pilot. It should not receive unredacted payroll, bank, fiscal, or close-certification payloads.

Helicone

Recommendation: useful if Stoquify wants gateway-style observability and routing quickly.

What it brings:

- LLM gateway, request logging, cost and latency tracking, caching, rate limits, fallbacks, prompt management, and agent debugging.
- Easy integration pattern for OpenAI-compatible SDKs and model traffic.

What it complicates:

- Gateway routing can become a production dependency for every model call.
- Data protection and redaction must be configured carefully.
- It overlaps with Langfuse in observability and prompt management.

Verdict: worth considering if the first priority is model gateway and cost control. If the first priority is evaluation and trace-quality loops, Langfuse may be the better first observability tool. Do not start with both unless there is a clear split.

LiteLLM

Recommendation: defer until multi-provider routing and budget enforcement matter.

What it brings:

- OpenAI-compatible interface across many providers.
- Proxy server for model routing, cost tracking, rate limits, budgets, and fallbacks.
- Useful abstraction if Stoquify wants to avoid provider lock-in.

What it complicates:

- It is another gateway service to deploy and secure.
- It does not replace product-level traces, approvals, redaction, or evidence binding.
- It can hide provider-specific behavior if not measured carefully.

Verdict: worth using later if Stoquify needs multi-model routing, provider fallback, and tenant or feature-level budgets. It is not the first dependency for the MVP.

5. Stoquify-Native Skills

Stoquify should own the domain skills. These should be versioned, tested, and connected to local services.

1. Trusted Context Resolver: builds the exact tenant, actor, role, locale, currency, date range, module entitlement, and route context for an agent run.
2. Permission and Entitlement Guard: filters tools, read models, fields, and actions through RBAC and module entitlement before the model sees them.
3. Evidence-Grounded Retrieval: retrieves only service-owned snapshot, proof trail, signal, and workbench data, then attaches source hashes and evidence grades.
4. Redaction and Safe Output: applies redaction-policy.service.ts before prompt construction and before final response rendering.
5. Freshness and Staleness Evaluator: uses snapshot freshness, source max update, and max age policies to mark answers fresh, stale, partial, or blocked.
6. Cash Exception Triage: turns drawer variance, pending payment, suspense, duplicate reference, and reconciliation failures into ranked next steps.
7. Reconciliation Match Suggestion: suggests match candidates without posting suspense or changing reconciliation status directly.
8. Inventory Risk Detector: identifies zero stock, negative stock, reorder risk, dead stock exposure, transfer gaps, and abnormal movement.
9. Replenishment Planner: drafts reorder suggestions from velocity, stock on hand, stock on order, supplier lead time, and cash exposure.
10. Stock-to-Cash Explainer: explains how stock movement, sales, cash collection, payments, ledger source links, and close readiness connect.
11. Close Blocker Navigator: explains close findings, checklist failures, unavailable evidence, waiver state, and required owner action.
12. Payroll Readiness Checker: summarizes employees, contracts, compensation, attendance, payment destinations, run readiness, declarations, and redactions.
13. Country Pack Provenance Resolver: explains country-pack version, review evidence, source hashes, statutory scenarios, and non-authoritative states.

14. Action Draft Builder: creates controlled draft payloads for existing service actions with idempotency key, risk level, permission, and approval policy.
15. Approval Router: routes drafts to the correct role, fresh-auth requirement, maker-checker policy, and due date.
16. Audit Narrative Builder: converts technical events into readable audit explanations without exposing hidden sensitive payloads.
17. Error and Exception Summarizer: converts safe error categories, correlation IDs, blockers, and retry states into user-facing next steps.
18. Agent Cost and Trust Monitor: detects cost spikes, repeated failures, low acceptance, stale answers, policy denials, and unsafe attempts.
19. Report Trust Export Certifier: verifies that exported agent reports include source links, redactions, evidence grade, freshness, and limitations.
20. Role Daily Brief Composer: turns signals and snapshots into owner, manager, accountant, cashier, HR, payroll, and finance officer briefs.

These skills are Stoquify's moat. They should not be outsourced to a generic framework.

6. First Agents To Launch

Command Agent

Users:

- Owner, manager, accountant, finance officer, branch manager, stockkeeper, HR/payroll lead, and auditor.

Daily jobs:

- Explain what changed today.
- Rank what needs attention.
- Show which action is blocked, who owns it, and what evidence supports it.
- Produce role-specific daily, weekly, and period-close briefs.

Read models:

- getTenantOperatingSnapshot
- getPaymentTruthSnapshot
- getInventoryCashSnapshot
- getCloseReadinessSnapshot
- business signals and action queue
- daily digest data
- owner war room and manager action center data
- proof trail service

Tools:

- read tenant snapshot
- read role action queue
- read business signals
- read proof trail
- draft action item assignment
- draft reminder
- explain blocker

UI entry points:

- /dashboard/daily-digest
- /dashboard/manager-action-center
- /dashboard/owner-war-room
- dashboard header command launcher

- module-level "Explain this" panels

Permissions:

- base permission: dashboard read
- filtered by each signal's requiredPermission
- module entitlement gate by source module
- redaction per sensitive category

Risk boundary:

- Read-only and draft-only in phase one.
- It may assign or route action drafts only through protected services.
- It must not resolve business actions without user confirmation.

Success metrics:

- daily active command users
- action queue open-to-resolved time
- accepted recommendation rate
- hidden or redacted signal count
- stale answer rate
- user feedback score

Minimum viable scope:

- A role-aware daily brief generated from snapshots, business signals, action queue, and proof trails.

Cash/Reconciliation Agent

Users:

- Finance officer, accountant, owner, manager, cashier supervisor, auditor.

Daily jobs:

- Explain where money is missing, unmatched, duplicated, pending, or blocked.
- Rank reconciliation failures by financial exposure and evidence quality.
- Draft suspense review tasks and match suggestions.
- Connect POS payments, provider references, cash drawer activity, settlement batches, and ledger source links.

Read models:

- payment-reconciliation-workbench.service.ts
- payment-truth-snapshot.service.ts
- CashPaymentHistoryWorkbench
- PaymentReconciliationWorkbench
- CashDrawerTransaction
- proof trails for payment transactions and reconciliation runs

Tools:

- read reconciliation workbench
- read payment truth snapshot
- read duplicate provider reference alerts
- read suspense-ready failures
- draft match suggestion
- draft suspense classification
- draft evidence request

UI entry points:

- /dashboard/finance/reconciliation
- /dashboard/finance/cash-command
- /dashboard/finance/cash-payment-history
- /dashboard/finance/payments
- /dashboard/cashDrawer

Permissions:

- payments.reconciliation.read
- finance.payments.read
- finance.cash-drawer.read
- stricter permissions for match, exception resolve, and suspense draft views

Risk boundary:

- No direct cash adjustment.
- No direct suspense posting.
- No direct ledger posting.
- No direct provider mutation.
- Only draft, recommend, and route.

Success metrics:

- time to identify unmatched funds
- open suspense amount
- duplicate reference detection rate
- reconciliation exception closure time
- accepted match suggestions
- false match suggestion rate

Minimum viable scope:

- Read-only triage and draft recommendations for duplicate references, open suspense, pending electronic payments, and cash variance.

Inventory/Replenishment Agent

Users:

- Stockkeeper, purchasing officer, manager, owner, cashier supervisor, accountant.

Daily jobs:

- Identify stockout risk, negative stock, zero stock, dead stock, overstock, transfer delays, and purchasing gaps.
- Explain cash tied in stock and stock-to-cash effect.
- Draft reorder suggestions and transfer review tasks.
- Surface stock risks before they become sales, cash, or close problems.

Read models:

- inventory-cash-snapshot.service.ts
- stock-to-cash-flow.service.ts
- inventory movement history
- purchase orders and item supplier links
- stock adjustments, transfers, counts, and item-level demand

Tools:

- read inventory cash snapshot
- read stock-to-cash data
- read item velocity and availability
- read supplier/item links

- draft reorder suggestion
- draft transfer suggestion
- draft count variance review

UI entry points:

- /dashboard/inventory
- /dashboard/inventory/items
- /dashboard/inventory/movements
- /dashboard/inventory/transfers
- /dashboard/purchase-orders
- /dashboard/finance/stock-to-cash

Permissions:

- inventory.read
- item read permissions
- purchasing read permissions for supplier/purchase order context
- write/draft permissions for purchase and transfer suggestions

Risk boundary:

- No direct stock adjustment.
- No direct write-off.
- No direct stock count certification.
- No direct purchase order approval.
- Draft only until human approval and service execution.

Success metrics:

- stockout reduction
- negative stock blocker reduction
- dead stock cash exposure reduction
- accepted reorder suggestions
- transfer delay resolution
- fewer close blockers caused by inventory data quality

Minimum viable scope:

- Read-only stock risk brief plus draft replenishment and count-review suggestions.

7. Platform Infrastructure Agents

Exception handling and action orchestration should be infrastructure because users do not need another chatbot. They need the system to preserve state, route work, and prevent unsafe automation.

Exception Orchestrator responsibilities:

- Convert safe errors, snapshot blockers, reconciliation failures, close findings, and workflow assurance incidents into normalized exception records.
- Attach severity, owner role, source module, evidence grade, source hash, and next action.
- Detect duplicate exceptions and stale unresolved work.
- Escalate critical exceptions to the correct role or route.

Action Orchestrator responsibilities:

- Convert an accepted recommendation into an AgentActionDraft.
- Validate permission, module entitlement, fresh-auth requirement, redaction state, and idempotency.
- Request approval when risk is medium or higher.
- Resume execution through service-owned APIs only.

- Record AgentStep, AgentApproval, evidence links, and audit events.

This maps well to the current business-signal and action-queue model. The existing action queue already has tenant filtering, permission filtering, severity, required permission, assigned role, evidence grade, blockers, and redactions. The agent layer should enrich and explain that queue, not replace it.

8. Safety And Authority Model

Agents must never do the following directly:

- Write directly to Prisma.
- Post ledger entries.
- File statutory reports.
- Change roles or permissions.
- Approve payroll.
- Release payroll payments.
- Certify close packs.
- Approve close waivers.
- Adjust cash.
- Write off stock.
- Approve stock counts.
- Approve purchase orders.
- Export sensitive reports without redaction and permission checks.

The controlled service-action model should be:

1. Agent reads only permission-filtered, module-entitled, redacted context.
2. Agent explains the issue and links evidence.
3. Agent drafts an action payload with risk level, required permission, source evidence, and idempotency key.
4. Stoquify validates the draft server-side using existing services.
5. If needed, Stoquify requests approval and fresh auth.
6. A human approves, rejects, or edits the draft.
7. The service executes the action through its existing protected path.
8. The runtime records the outcome, evidence, audit event, and feedback.

This is especially important for ledger and close. `posting.service.ts` already has accounting invariants, idempotency, source links, transactions, and ledger audit events. Agents must call approved service actions through that path only. Similarly, close assurance already has controls for assignment, waiver fresh auth, certification availability, and evidence coverage. Agents should explain and draft, not certify.

9. UI/UX Integration

The UI should feel like Stoquify became more operationally intelligent, not like a chat widget was pasted onto the product.

Recommended surfaces:

- Command strip: a small role-aware command summary at the top of command surfaces.
- Module assistant panel: right-side panel for "explain this", "why is this blocked", "what should happen next", and "show evidence".
- Evidence drawer: reuse and extend `ProofTrailDrawer` for agent evidence links.
- Action draft card: shows recommendation, required permission, affected records, evidence grade, risk level, approval state, and service path.
- Approval timeline: shows requested, reviewed, approved/rejected, executed, failed, or expired.
- Exception queue: integrated into Manager Action Center and Assurance Control Tower.

- Daily brief: role-specific brief in Daily Habit Digest and Owner War Room.
- Inline evidence badges: use evidence grade and freshness in agent answers.
- Redaction notices: show masked/redacted categories without leaking protected data.

Avoid:

- Floating chat everywhere.
- Agent messages that cannot show evidence.
- Agent actions hidden behind vague "Done" states.
- Large AI-branded hero surfaces inside operational modules.
- Model-visible tools that are broader than the current user and tenant context.

The current UI already has the right primitives: DailyHabitDigestDashboard, FinanceCommandCenterDashboard, command center primitives, evidence badges, proof trail drawer, dashboard route states, and finance command normalization. The agent UI should extend those surfaces.

10. Implementation Roadmap

First 30 days

Goal: safe read-only runtime.

- Add minimal agent runtime schema: definitions, runs, steps, evidence links, feedback, cost ledger.
- Add services/agents folder with context resolver, tool registry, run logger, evidence binder, and redaction wrapper.
- Add internal tool wrappers for snapshots, business signals, action queue, proof trails, payment reconciliation workbench, and inventory cash snapshot.
- Add Command Agent in read-only mode for internal users.
- Add role daily brief composer using existing business signals and action queue.
- Add a small admin/dev console to inspect agent runs and policy denials.
- Add tests for tenant isolation, permission filtering, redaction, stale snapshots, and no-write enforcement.

Days 31-60

Goal: read-and-draft for cash and inventory.

- Add Cash/Reconciliation Agent read-and-draft mode.
- Add Inventory/Replenishment Agent read-and-draft mode.
- Add AgentActionDraft and AgentApproval.
- Draft action types: assign action item, request evidence, suggest match, suggest suspense classification, suggest reorder, suggest transfer review, suggest stock count review.
- Add approval timeline UI.
- Add feedback capture: accepted, rejected, stale, wrong, unsafe, not relevant.
- Start measuring time saved and recommendation acceptance.

Days 61-90

Goal: controlled low-risk execution.

- Allow only low-risk service actions with idempotency.
- Introduce one durable execution layer if approval wait/retry/resume is needed.
- Add tool approval validation with signed input hash.
- Add Langfuse or Helicone for redacted traces, costs, latency, tool outcomes, and evaluations.
- Add platform assurance checks: unsafe tool attempts, cost spikes, stale outputs, low acceptance, repeated failures, and policy denials.

- Keep ledger, statutory, payroll approval, role changes, close certification, cash adjustment, and stock write-off human-owned.

After 90 days

Goal: expand to deeper finance, payroll, compliance, and customer success.

- Purchasing/AP Agent for PO delays, supplier exposure, AP exceptions, and fraud-control readiness.
- Close and Compliance Agent for close blockers, evidence gaps, waiver routing, and compliance readiness.
- Payroll Readiness Agent for HR/payroll inputs, attendance, payment destinations, declarations, and close exposure.
- Customer Success/Adoption Agent for tenant onboarding, module adoption, stuck workflows, and support deflection.
- Convert accepted recommendations into product analytics and workflow intelligence.

11. Least-Code Highest-Leverage Strategy

The best implementation uses existing Stoquify assets:

- Use snapshots as agent context instead of building new queries.
- Use business signals as agent attention objects instead of inventing another alert model.
- Use action queue as the shared work-routing substrate.
- Use proof trails as evidence links.
- Use RBAC and module entitlement before model context construction.
- Use redaction policies before prompts and final answers.
- Use protected service actions for every mutation.
- Use existing dashboard routes for UI entry points.
- Use existing release gates and tests to enforce boundaries.

The first implementation slice can be surprisingly small:

- One services/agents runtime service.
- One tool registry.
- Five read-only tools.
- One Command Agent.
- One proof/evidence binding helper.
- One run logger.
- One UI panel inside Daily Digest or Manager Action Center.
- Tests that prove no direct Prisma write tool exists.

This is the least-code path because Stoquify already has the hard domain model. The agent runtime should be a thin decision and explanation layer.

12. Risks And Tradeoffs

Architectural risk: parallel business logic

Risk: agent tools duplicate service logic and create inconsistent answers.

Mitigation: all tools must wrap existing service-owned read models or protected actions. No raw Prisma tools.

Security risk: prompt data leakage

Risk: sensitive payroll, bank, fiscal, compliance, or audit data enters prompts.

Mitigation: apply redaction before prompt construction and store only safe summaries in traces.

Authority risk: unsafe automation

Risk: users trust an agent that silently changes cash, stock, payroll, ledger, or permissions.

Mitigation: read/draft first, human approval, fresh auth, maker-checker, idempotency, and service execution only.

Compliance risk: non-authoritative OHADA advice

Risk: the agent presents statutory or accounting interpretations as law.

Mitigation: country-pack provenance, expert-reviewed rules, citations to internal configuration, and clear non-authoritative states.

Product risk: generic chatbot fatigue

Risk: users ignore a generic assistant because it does not fit work.

Mitigation: role-specific briefs, exception queues, evidence drawers, action drafts, and module surfaces.

Cost risk: uncontrolled model usage

Risk: recurring daily/hourly agents create high LLM spend.

Mitigation: cache snapshots, summarize read models, route by risk, use smaller models for classification, track cost per tenant/module, and add budgets.

Trust risk: stale or wrong recommendations

Risk: a stale snapshot produces bad advice.

Mitigation: freshness gates, source max update, source hash, stale labels, feedback, and model evaluations.

Operational risk: durable workflow sprawl

Risk: too many background workflows become hard to debug.

Mitigation: choose one durable execution layer, version functions, require idempotency, and centralize workflow monitoring.

13. Success Criteria

Product success:

- 60 percent or more of pilot users open the Command Agent at least 4 days per week.
- 40 percent or more of high-confidence recommendations are accepted or routed.
- Users report reduced time to understand daily priorities.

Cash and reconciliation success:

- Open suspense amount declines.
- Time to identify duplicate provider references declines.
- Reconciliation exception resolution time declines.
- False match suggestion rate remains below an agreed threshold.

Inventory success:

- Negative stock blockers decline.
- Zero stock incidents decline.
- Accepted reorder suggestions increase.
- Dead stock cash exposure becomes visible and actionable.

Trust success:

- Every recommendation has evidence grade, freshness, source module, and source hash.
- No agent output bypasses redaction policy.
- No direct Prisma write or direct ledger/statutory/payroll/permission/stock/cash mutation tool exists.
- Agent policy incidents are logged and reviewable.

Engineering success:

- Agent tools are service-owned and schema-versioned.
- Tests cover tenant isolation, permission filtering, redaction, no-write rules, approval requirements, and idempotency.
- Release gates include agent safety checks before production rollout.

14. Final Recommendation

Stoquify should pursue the proposal. It is strategically aligned, technically feasible, and unusually high-leverage because the codebase already has the ingredients that generic SMB tools do not: module entitlement, RBAC, workflow assurance, snapshots, business signals, action queues, payment reconciliation, inventory cash truth, close assurance, payroll command read models, evidence grades, proof trails, redaction, fresh auth, ledger idempotency, and release gates.

The first implementation slice should be:

1. Add the shared agent runtime schema and services.
2. Add a TypeScript-native agent loop using AI SDK or equivalent.
3. Wrap existing snapshot, signal, action queue, proof trail, payment reconciliation, and inventory cash read models as read-only tools.
4. Launch the Command Agent internally as read-only.
5. Add Cash/Reconciliation and Inventory/Replenishment in read-and-draft mode.
6. Add approval and idempotent service-action execution only after real usage validates the workflows.

This path positions Stoquify at the helm of SMB tools because it turns the product into a frequent-use operating system: not just records, not just dashboards, not just AI chat, but trusted daily action with evidence.

Proposed File And Module Layout

Recommended initial additions:

```
services/agents/
  agent-context.service.ts
  agent-tool-registry.service.ts
  agent-runner.service.ts
  agent-evidence.service.ts
  agent-redaction.service.ts
  agent-action-draft.service.ts
  agent-approval.service.ts
  agent-observability.service.ts
  agent-contracts.ts
  __tests__/
    agent-context.service.test.ts
    agent-tool-registry.service.test.ts
    agent-redaction.service.test.ts
    agent-no-direct-write-boundary.test.ts

components/agents/
  AgentCommandPanel.tsx
  AgentEvidenceDrawer.tsx
  AgentActionDraftCard.tsx
  AgentApprovalTimeline.tsx
```

```
app/[locale]/(dashboard)/dashboard/agent-runs/  
page.tsx
```

Recommended first tools:

```
readTenantOperatingSnapshot  
readActionQueue  
readProofTrail  
readPaymentReconciliationWorkbench  
readInventoryCashSnapshot  
draftActionItemAssignment  
draftEvidenceRequest  
draftReconciliationSuggestion  
draftReplenishmentSuggestion
```

Prohibited tools:

```
prismaRawQuery  
prismaCreate  
prismaUpdate  
postLedgerEntryDirectly  
fileStatutoryReport  
changeUserPermissions  
approvePayrollRun  
certifyClosePack  
adjustCashDrawer  
writeOffStock  
approveStockCount
```

Verification Plan For Implementation

Suggested commands once implementation begins:

```
npm run prisma:validate  
npm run typecheck  
npm run lint  
npm run service:boundary:fail  
npm run workflow:assurance:runtime-check  
npm run payment:cash-truth:gate  
npm run inventory:boundary:fail  
npm run ledger:close-truth:gate  
npm run report:trust:export:gate  
npm test -- --runInBand services/agents
```

Suggested manual checks:

- Login as owner, accountant, cashier, stockkeeper, HR/payroll manager, and auditor.
- Confirm each role sees different agent context and redactions.
- Confirm every recommendation links to proof or source data.
- Confirm stale snapshots produce caution states.
- Confirm prohibited tools are not registered.
- Confirm action drafts require the same protected service path as manual actions.
- Confirm approval replay revalidates permissions, module entitlement, and input hash.

Bottom Line

The proposal is worth doing. Stoquify should not compete by adding "AI chat." It should compete by becoming the safest and most useful daily command layer for SMB work. The moat is evidence-backed execution across POS, inventory, purchasing, payments, reconciliation, accounting close, payroll, compliance, RBAC, audit trails, and workflow assurance. The agent runtime should make that moat visible, usable, and habit-forming.